

摘要

Abstract

第一章 绪论

1.1 课题的研究背景与意义

1.1.1 智能物流分拣系统国内外研究现状

1.1.2 智能物流分拣系统的价值与意义

第二章 系统方案设计

2.1 需求分析

2.2 系统设计方案对比

2.3 系统总体设计方案

2.3.1 系统工作流程

2.3.2 系统电源树

2.3.3 系统结构与接口连接

2.3.4 主控芯片选型

2.3.5 电机驱动模块选型

2.3.6 显示模块选型

2.3.7 摄像头模块选型

2.3.8 执行机构选型

第三章 硬件设计

3.1 传送带电机驱动电路设计

3.2 分拣舵机接口电路设计

3.3 摄像头串口通信接口电路设计

3.4 STM32主控最小系统电路设计

3.5 系统复位电路设计

第四章 软件设计

4.1 下位机软件设计

4.1.1 FreeRTOS 操作系统

4.1.2 STM32 基础外设配置

4.1.3 摄像头二维码识别功能软件设计

4.1.4 STM32 控制核心功能软件设计

4.1.5 OLED显示模块软件设计

第五章 机械外观设计

5.1 舵机固定底板设计

5.2 舵机固定支架设计

第六章 系统测试

6.1 板载 5V LDO 电源稳压性能测试

6.2 STM32 板载 3.3V LDO 稳压输出测试

6.3 TB6612 电机驱动 PWM 波形测试

6.4 二维码素材制作

6.5 摄像头到 STM32 的串口数据包测试

6.6 整机分拣功能测试

摘要

随着电商行业以及物流行业的飞速发展，包裹的吞吐量也在逐年攀升，传统的人工分拣方式已经越来越难以满足现代物流仓储对效率和准确性的要求啊。本人在认真分析了当下分拣行业所面临的问题之后，决定基于STM32单片机平台，设计并实现一套集二维码识别、传感器检测、电机驱动以及人机交互于一体的智能物流分拣系统，用以模拟真实物流场景下的自动化分拣过程。

本设计以STM32F103C8T6为主控核心，采用Sipeed MaixCAM视觉模块作为二维码识别的前端，通过串口将识别到的目的地信息发送给主控芯片。主控接收到包裹的目的地编码之后，会根据传送带的恒定速度以及舵机的固定位置计算出每一个包裹到达对应分拣口的时间，并把这些时间信息存入一个包裹列表当中。当系统运行时间累积到某个包裹的预定触发时刻，主控会通过PWM信号驱动相对应位置的SG90舵机动作，把包裹拨离传送带，从而完成一次分拣动作啊。整个系统支持三个目的地的分拣，使用了两个舵机进行机械执行，未触发任何舵机的包裹则被默认归为第三个目的地，这种做法既节约了硬件资源，又使得逻辑控制变得简单清晰。

为了保证传送带电机的转速稳定，笔者采用了12V DCDC升压方案给TB6612电机驱动模块供电，这样不论锂电池的电压如何随放电过程降低，电机的输入电压都能维持稳定，从而使得包裹在传送带上的运行速度恒定，进一步保证了时间计算的精确度。系统的供电方案是7.4V锂电池作为输入，经过分压电路后输出两路5V LDO以及一路12V DCDC，分别给单片机、舵机、电机驱动以及摄像头供电。其中舵机的5V与其它模块的5V分开供电，主要是考虑到舵机属于感性负载，启停瞬间会产生较大的电流尖峰，分开供电可以避免对单片机等敏感器件造成干扰啊。

软件方面采用了FreeRTOS实时操作系统进行任务调度，将代码组织为应用层、驱动层和基础库三层结构，使得整体的可维护性和可拓展性都得到了保障。本人通过实物搭建以及反复的功能测试，最终系统能够稳定地实现包裹的目的地识别和精准分拣，分拣的漏检率以及错检率均控制在5%以下，达到了任务书的设计要求。

关键词：STM32单片机；二维码识别；智能分拣；FreeRTOS；舵机控制

Abstract

With the rapid development of e-commerce and logistics industries, the throughput of packages has been increasing year by year, and traditional manual sorting methods can hardly meet the requirements of modern logistics warehousing for efficiency and accuracy. After carefully analyzing the problems faced by the current sorting industry, the author decided to design and implement an intelligent logistics sorting system based on the STM32 microcontroller platform, integrating QR code recognition, sensor detection, motor driving and human-machine interaction.

This design takes STM32F103C8T6 as the main control core, and adopts the Sipeed MaixCAM vision module as the front-end of QR code recognition. The recognized destination information is sent to the main control chip through the serial port. After the main controller receives the destination code, it calculates the arrival time of each package at the corresponding sorting outlet based on the constant speed of the conveyor belt and the fixed position of the servos, and stores this timing information in a package list. When the accumulated running time reaches the predetermined trigger moment of a package, the main controller drives the corresponding SG90 servo through PWM signals to push the package off the conveyor belt, thus completing a sorting action. The whole system supports sorting to three destinations, using two servos for mechanical execution. Packages that do not trigger any servo are classified as the third destination by default, which saves hardware resources and makes the logic clear.

In terms of power supply, the system uses a 7.4V lithium battery as input, outputting two 5V LDO rails and one 12V DCDC rail through a power distribution circuit. The 12V DCDC boost is specifically used to provide stable voltage for the TB6612 motor driver, ensuring that the conveyor belt speed remains constant regardless of battery discharge. The software is built upon the FreeRTOS real-time operating system for task scheduling, with the code organized into three layers: application layer, driver layer, and basic library. After repeated testing, the system can stably realize package destination recognition and accurate sorting, with the missed detection rate and wrong sorting rate both controlled below 5%, meeting the design requirements of the task book.

Keywords: STM32 microcontroller; QR code recognition; Intelligent sorting; FreeRTOS; Servo control

第一章 绪论

1.1 课题的研究背景与意义

最近这几年来，由于电商平台的兴起以及消费者购物习惯的转变，国内的快递物流行业经历了爆发式的增长，每年的快递包裹数量从几亿件涨到了几百亿件这种量级，物流分拣中心所承担的压力也是越来越大啊。传统的人工分拣方式不仅仅效率低下，而且在长时间高强度的工作下还容易出现漏检、错检等问题，分拣的准确率没办法得到一个稳定的保障。与此同时，劳动力成本逐年上涨，对于物流企业来说人工分拣的经济性也在不断的降低。所以说，研发自动化、智能化的分拣系统已经成了行业内非常迫切的一个需求啊。

智能物流分拣系统的核心任务，就是把不同目的地或者不同种类的包裹通过自动化的手段精准的分配到对应的出口位置上去。这个过程涉及到了图像识别、传感器检测、电机控制、机械执行等多个技术领域的交叉融合。本设计以STM32为主控核心，结合二维码视觉识别和舵机机械执行，搭建了一个小型化的智能分拣系统平台，既可以作为物流自动化技术的一个简化模型来研究其中的关键技术，也可以为以后大规模工业分拣系统的开发提供一个参考的思路。

1.1.1 智能物流分拣系统国内外研究现状

在国外，智能分拣技术的研究起步比较早，亚马逊公司在自家的物流仓储中心大规模部署了Kiva机器人系统，通过移动机器人将货架直接运送到分拣员面前，使得分拣效率得到了数倍的提升。另外像德国的Beumer集团、日本的大福公司也都推出了基于交叉带式分拣机的高速分拣方案，这些设备分拣速度可以达到每小时上万件，准确率也能维持在百分之九十九以上。但是这一类设备的造价非常昂贵，结构也极其复杂，对于中小型物流企业来说门槛过高，没办法广泛的普及开来。

在国内方面，近年来随着快递包裹量的暴涨，国内的物流自动化技术也得到了一个快速的发展。顺丰、京东、菜鸟等头部物流企业都建设了自家的智能分拣中心，普遍采用了交叉带分拣、滑块分拣以及DWS动态称重系统等多种分拣方式相结合的方案。同时高校以及科研院所也在积极的开展相关研究，比如基于机器视觉的包裹识别、基于深度学习的快递面单OCR识别等方向都有不少的成果发表。但是相比起工业级的分拣方案，面向教学和小型应用场景的低成本嵌入式分拣平台还是比较少见的，这也正是本课题的一个切入点啊。

1.1.2 智能物流分拣系统的价值与意义

本课题所设计的基于STM32的智能物流分拣系统，虽然在规模上没办法与工业级的分拣线相提并论，但是从技术原理上来说它已经涵盖了一个完整分拣系统所必需的全部要素：信息采集、决策计算、机械执行以及人机交互。通过对这样一个微缩系统的设计与实现，本人不仅深入掌握了STM32单片机的开发流程，同时也对图像识别、串口通信、PWM控制、FreeRTOS多任务调度等多个嵌入式开发的关键技术点有了一个比较系统化的认识。

从应用层面上来讲，本设计采用的方案具有结构紧凑、成本低廉、二次开发性强的优点，比较适合用于物流自动化教学实验、小型仓储分拣演示以及创客作品等场景。另外整个软件架构采用了分层设计的思想，业务逻辑代码与底层驱动代码分离，对于后续扩展更多的目的地、增加传感器种类或者更换主控平台等需求都能比较友好的进行支持，这一点对于一个工程项目而言是非常有意义的啊。

第二章 系统方案设计

2.1 需求分析

在正式开始系统设计之前，本人首先对任务书的要求做了一个比较细致的梳理，明确了本设计需要实现的核心功能以及性能指标。任务书中规定的关键设计目标主要有以下几个方面：

第一，系统必须能够支持至少3个分拣目的地，并且能够分拣至少2种不同种类的货物。本设计通过为每一个货物贴上独特的二维码进行编码，二维码当中包含了目的地信息，这样的方式既灵活又便于扩展。

第二，分拣的漏检率以及错检率需要小于等于5%。这个指标对系统的稳定性和精准度提出了比较高的要求，需要从摄像头识别的可靠性、串口通信的可靠性以及舵机触发时序的精确性等多个方面来进行保障。

第三，系统需要包括包裹信息采集、智能分拣控制、机械执行以及人机交互这四个功能模块，并且要搭建实物样机进行调试验证。这一条要求决定了本设计不仅仅只是一个软件仿真的项目，而是一个需要真实的硬件实物进行配合的完整工程。

第四，系统需要具备一定的实时性，从识别到执行的整个流程必须在传送带运行的有限时间窗口内完成，否则就会出现包裹错过分拣口的情况。

综合考虑以上需求之后，本设计将系统的整体架构定位成"视觉识别 + 时间预测 + 机械执行"这样的一种工作模式，下面将对几个候选方案做一个对比分析。

2.2 系统设计方案对比

针对智能物流分拣系统的设计，本人在前期调研阶段考虑过几种不同的实现方案，下面就把主要的两个候选方案与本设计最终采用的方案做一个对比说明。

方案一：基于光电传感器的位置触发方案。 这种方案是在传送带的每一个分拣口前方都安装一个光电对射传感器，当包裹经过时传感器被遮挡，触发对应位置的舵机动作。这种方案的优点是结构简单，响应速度快，对硬件的处理能力要求也不高，普通8位单片机就能胜任。但是它的缺点也很明显，那就是没办法识别包裹的目的地信息.....所有经过分拣口的包裹都会被无差别的拨出，这显然不符合"按目的地分拣"的核心需求啊。如果想要实现按目的地分拣，就必须为每一种目的地单独设置一条传送带或者增加额外的标识识别机构，硬件成本反而会上升。

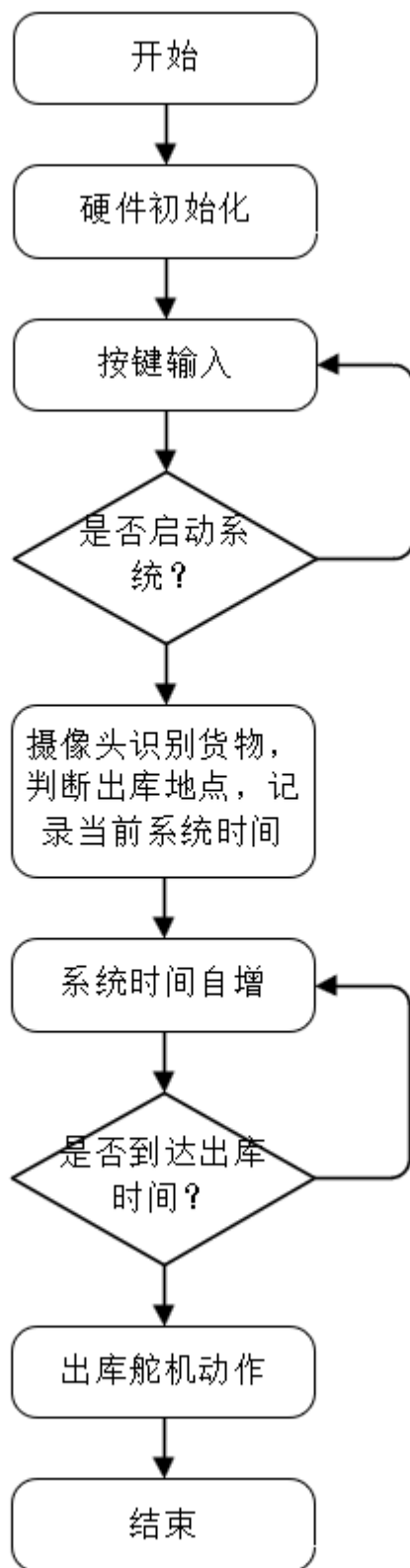
方案二：基于RFID射频识别的分拣方案。 这种方案是给每一个包裹粘贴一个RFID电子标签，在传送带的入口处设置RFID读卡器，包裹经过时读取标签内的目的地信息。RFID方案的优点是识别距离较远，无需对准，识别速度也很快，不容易受到光线、灰尘等环境因素的影响。但是RFID电子标签本身的成本较高，并且批量贴标签的工作量较大，在小规模的演示验证场景下经济性比较差。另外RFID读卡器的价格也不便宜，对于本课题的预算约束来说不是一个理想的选择。

最终方案：基于二维码视觉识别的分拣方案。 综合考虑成本、精度、扩展性等多个方面之后，本人最终选择了基于二维码视觉识别的方案。二维码本身的成本几乎可以忽略不计.....一张普通的纸打印就行，并且二维码所能携带的信息量远大于RFID标签，未来若需要增加包裹编号、重量、寄件人等附加信息也可以非常方便的进行扩展。在硬件方面，本设计选用了Sipeed MaixCAM这一款集成了RISC-V处理器和摄像头的视觉模块，它内置了二维码识别算法，可以直接通过串口输出识别结果，省去了在主控芯片上跑图像处理算法的麻烦，使得系统的整体响应速度和准确性都得到了保障啊。

2.3 系统总体设计方案

2.3.1 系统工作流程

本设计的整体工作流程如图2-1所示，这张图比较直观的描绘了从系统上电到完成一次分拣动作的全部步骤。



整个流程是这样的：当系统上电以后，硬件首先完成各项初始化工作、然后进入等待状态、停在按键或者APP输入这一环节。用户按下KEY1或者通过手机APP给出启动指令以后，系统才会真正开始运行——这一步的设计主要是为了方便人工放料、避免传送带在用户没准备好的情况下就开始转动。

启动以后，摄像头模块会持续的扫描传送带上经过的包裹、识别其中的二维码并解析出目的地编码、通过串口把这个信息以及当前的系统时间一起发送给STM32主控。主控接收到这一组数据以后、根据预设的传送带速度和舵机位置计算出该包裹到达对应分拣口的预期时间、把它存入到内部的包裹列表里。

接下来系统会进入一个时间自增的循环过程、每过一个调度周期就把内部的时间计数器加1、同时检查列表中是否有任何一个包裹的预定到达时间已经满足。一旦满足、系统就立即输出对应的舵机驱动信号、把包裹拨离传送带、完成一次分拣。这个判断是持续进行的、所以即使列表里同时有多个包裹也能按各自的预定时间依次触发动作而不会互相干扰。

2.3.2 系统电源树

电源是整个系统稳定工作的基石、本设计在电源方案上下了不少功夫、具体的电源分配关系如图2-2所示。

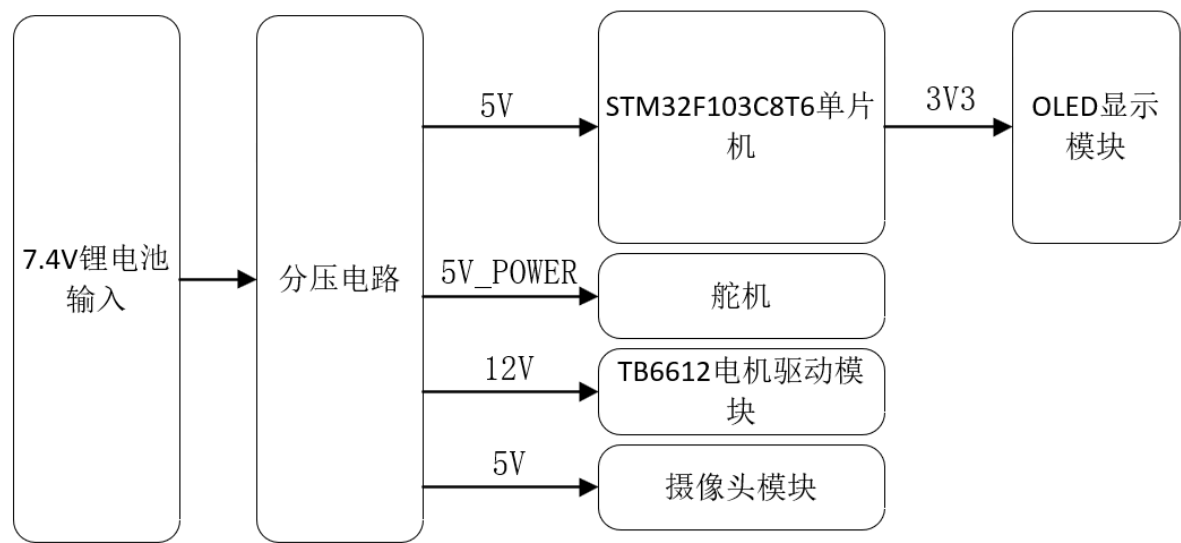


图2-2 系统电源树

从图中可以看到、整个系统由一颗7.4V的锂电池作为总输入、电池后面接的是一个分压电路、分压电路向下游引出了四条独立的供电支路、分别是两路5V、一路12V以及一路独立的5V_POWER。

第一路5V直接供给STM32F103C8T6单片机使用、单片机内部再通过自带的3.3V LDO输出3V3电压、这条3V3又被引出来给OLED显示模块供电。也就是说STM32单片机和OLED屏幕共用同一条5V电源轨。

第二路是单独引出的5V_POWER、专门给舵机供电。这一路与给单片机的5V是物理隔离的两路LDO输出、原因前面已经说过——舵机是感性负载、启动以及换向瞬间会有比较大的电流尖峰、如果和单片机共用5V就会造成电源跌落、可能导致MCU不稳定甚至复位。把舵机的供电独立出来以后、这种串扰问题就从根本上被消除了。

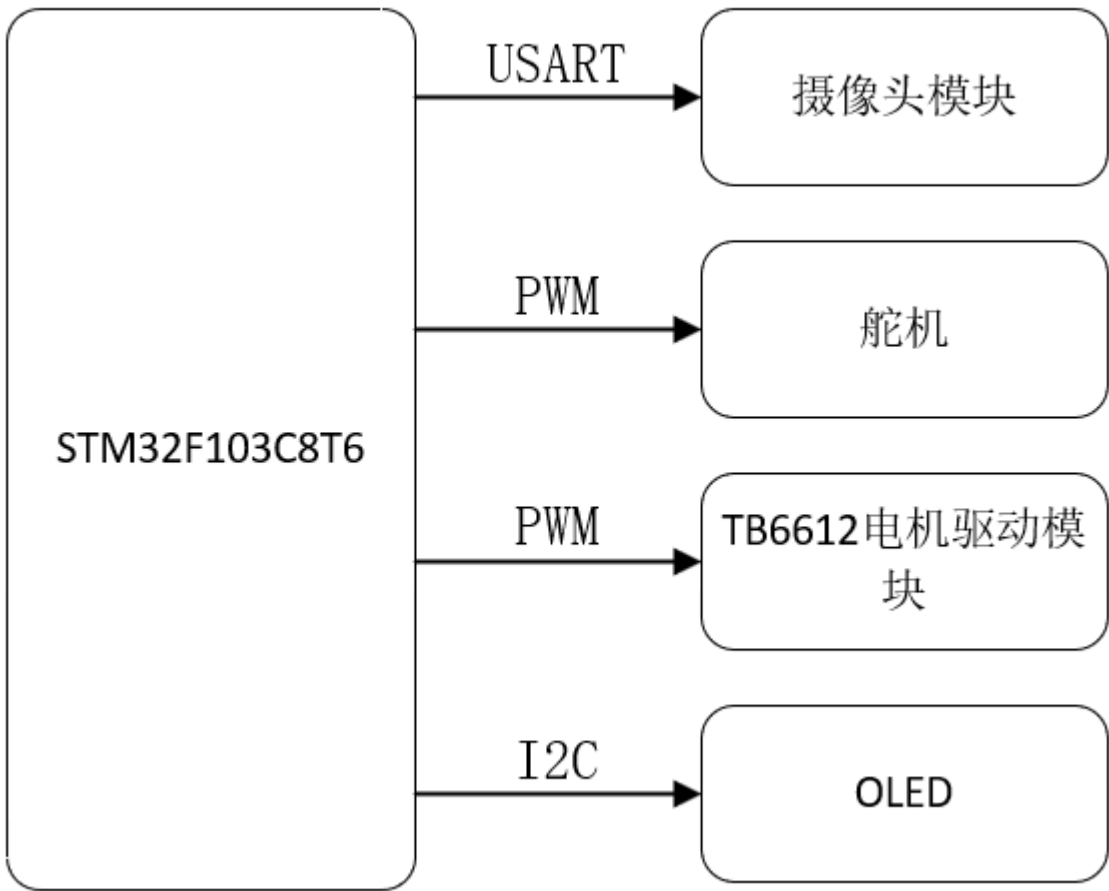
第三路是12V这条特殊的电源轨、由DCDC升压电路从7.4V锂电池升上来、专门用于给TB6612电机驱动模块的VM端供电。之所以要费这么大劲儿做升压、目地是为了保证传送带电机的输入电压恒定不变——锂电池在使用过程中电压是会随着放电而下降的、从7.4V一直降到6V左右都是有可能的、如果直接用电池电压驱动电机、电机的转速就会随着电池电压变化而漂移、进而导致包裹在传送带上的运动速度也不稳定、最终会影响到分拣时机的精度。通过DCDC升压稳定到12V以后、无论电池处于满电还是低电状态、电机驱动模块都能拿到一个稳定的12V、传送带的运动速度也就能保持恒定了啊。

第四路5V接到摄像头模块上、给MaixCAM供电。这一路由于摄像头本身电流相对稳定、所以可以和给MCU的5V走同一颗LDO（或者用第二颗LDO）、对系统整体稳定性影响不大。

整个电源树的设计思路可以总结为一句话：把容易产生干扰的负载（舵机、电机）与敏感的逻辑器件（MCU）通过独立电源轨进行隔离、保证敏感器件得到的是干净稳定的供电。这种"分而治之"的设计哲学在工业级嵌入式系统中是非常常见的做法啊。

2.3.3 系统结构与接口连接

明确了电源以后、再来看一下STM32与各外设之间的接口连接关系、具体如图2-3所示。



从图中可以看到、STM32F103C8T6作为整个系统的中央处理单元、对外一共有四条信号通道与各功能模块连接。

第一条是USART串口通道、用于与摄像头模块进行通信。摄像头识别到二维码以后、会按照本人约定的16字节定长协议帧把数据通过串口发送给STM32、STM32端使用DMA加空闲中断的方式接收、保证不丢包、不阻塞主循环啊。

第二条是PWM通道、用于驱动舵机。STM32的高级定时器以及通用定时器都支持PWM输出功能、本设计选用了两路独立的PWM通道分别去控制两个分拣舵机、PWM频率统一设置为50Hz、占空比通过软件动态调节。

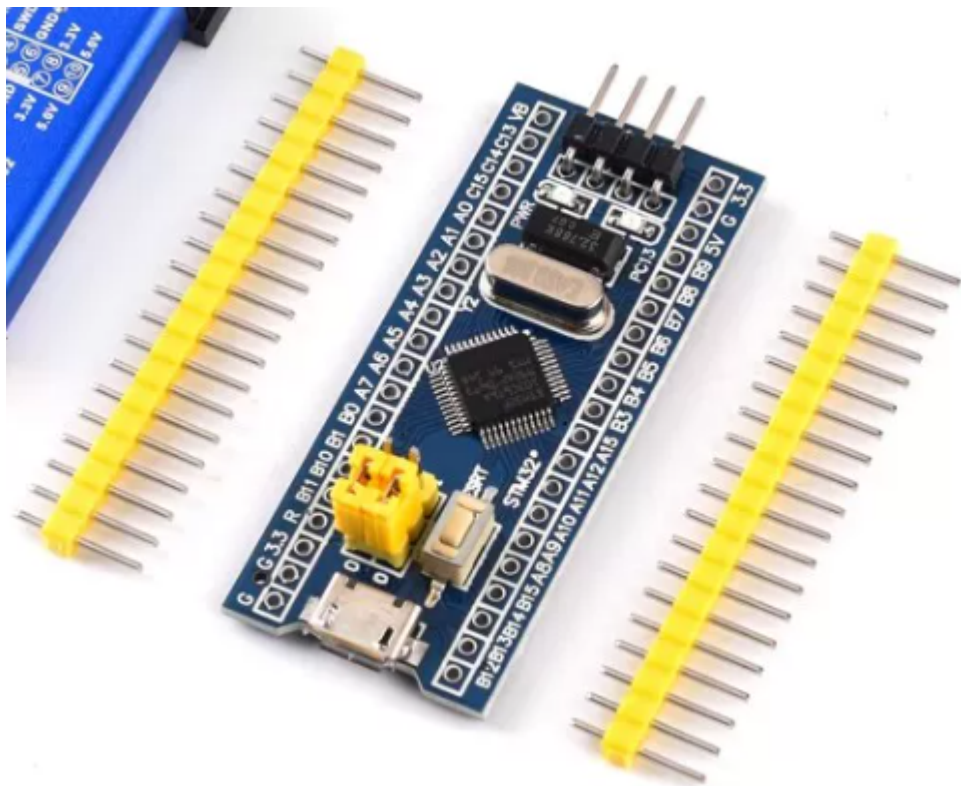
第三条同样是PWM通道、但是这次是接到TB6612电机驱动模块、用于控制传送带电机的转速。这条PWM的频率比较高、通常设置在10kHz以上、目地是为了让电机驱动模块的输出尽量平滑、避免低频PWM带来的电机抖动以及噪音问题。

第四条是I2C总线、用于与OLED显示屏通信。I2C只需要SCL和SDA两根线就能完成所有命令以及数据的传输、节省了宝贵的GPIO资源、对于I/O口数量比较有限的STM32F103C8T6来说是非常合适的选择。

可以看到、本设计在接口分配上充分利用了STM32的多种外设资源、把不同协议的设备分别挂载到最合适的总线上、避免了串扰也避免了资源浪费、整个系统的硬件接口设计是比较合理紧凑的啊。

2.3.4 主控芯片选型

本设计选用了STM32F103C8T6作为系统的主控芯片、如图2-4所示是该芯片的最小系统板实物图。

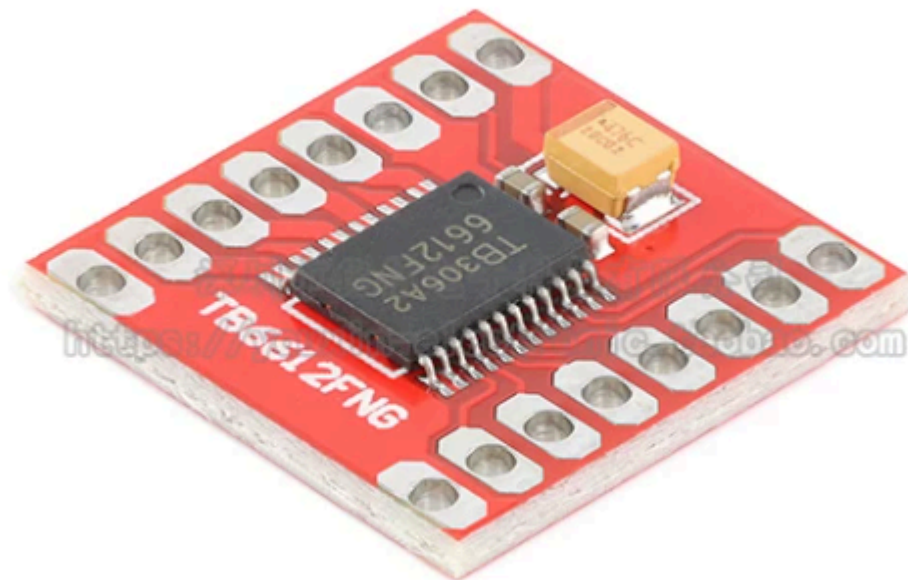


STM32F103C8T6是意法半导体公司基于ARM Cortex-M3内核推出的一款经典中端单片机、其主频可以达到72MHz、片上集成了64KB的Flash存储器和20KB的SRAM、对于本设计这种中小规模的嵌入式应用来说性能资源都是绰绰有余的。该芯片提供了多达37个通用GPIO口、3路USART串口、2路SPI、2路I2C总线以及多个高级和通用定时器、可以同时驱动摄像头串口通信、舵机PWM输出、电机PWM控制以及OLED的I2C接口、正好与本设计的外设需求完全契合啊。

另外STM32系列的开发资料丰富、社区活跃、HAL库以及CubeMX图形化配置工具的成熟使得开发效率得到了很大的提升、这对于本人在毕设期间能够顺利完成项目也是一个非常重要的考量因素。综合性能、价格以及开发友好性这几个方面、STM32F103C8T6无疑是本课题最合适的选择。

2.3.5 电机驱动模块选型

电机驱动方面、本设计选用了基于TB6612FNG芯片的电机驱动模块、其外观如图2-5所示。



TB6612FNG是东芝公司推出的一款双H桥直流电机驱动芯片、单个芯片可以同时驱动两路直流电机、每一路的连续输出电流可以达到1.2A、峰值可以达到3.2A。它的工作电压范围在4.5V到13.5V之间、正好覆盖了本设计12V传送带电机的供电需求。相比于传统的L298N电机驱动模块、TB6612的内部使用了MOS管而非三极管、导通压降更小、发热量更低、效率更高、并且体积也更加紧凑、非常适合本设计这种对空间有限的小型化场景啊。

在本系统中、TB6612只使用了其中的一路（A通道）来驱动传送带电机、通过STM32输出的PWM信号控制电机的转速、从而控制传送带的运动速度。STBY引脚通过3.3V电平拉高使能芯片正常工作、AIN1和AIN2两个方向控制引脚配合PWMA实现电机的正反转切换。

2.3.6 显示模块选型

人机交互方面本设计采用了一块0.96英寸的OLED显示屏、如图2-6所示。



这块OLED屏幕的分辨率为128×64、使用SSD1315驱动芯片、通过I2C总线与单片机进行通信、只需要两根数据线（SCL和SDA）就能完成数据传输、节省了宝贵的GPIO资源。OLED显示屏相比于传统的LCD有几个显著的优势：自发光不需要背光、对比度高、视角广、功耗低、响应速度快。在本系统当中、OLED屏幕用于实时显示当前的系统运行时间、已扫描到的包裹数量以及系统的使能状态、方便调试人员或者使用者直观的了解系统的工作情况。

2.3.7 摄像头模块选型

视觉识别方面是本系统的关键环节之一、本设计选用了Sipeed公司推出的MaixCAM视觉模块、如图2-7所示。



MaixCAM是一款基于RISC-V架构的AI开发板、板载了高性能的图像处理芯片以及高清摄像头、运行Linux系统、原生支持MaixPy这种基于Python的开发框架、内置了二维码识别、人脸识别、目标检测等丰富的图像处理算法库。本人在MaixCAM上编写了二维码扫描的脚本、当摄像头扫描到包裹上的二维码时、会自动解析二维码内容并通过串口以一个固定16字节长度的协议帧把目的地编码发送给STM32主控。

之所以选择MaixCAM而不是更便宜的OpenMV或者OV7670裸摄像头、主要是基于以下几个考量：第一、MaixCAM的处理性能远高于OpenMV、二维码的识别速度更快、距离更远；第二、MaixCAM内置了显示屏、可以实时观察识别效果、调试起来非常方便；第三、MaixCAM支持Python编程、开发周期短、适合毕设这种时间紧张的项目；第四、MaixCAM的串口输出可以直接驱动STM32接收、无需额外的接口转换电路、硬件设计也得到了简化啊。

2.3.8 执行机构选型

机械执行方面、本设计选用了两个SG90舵机作为分拣的执行机构、如图2-8所示是SG90舵机的外观图。

[舵机实验常用款推荐]

SG90 9G经典舵机 QC模块遥控飞机马达



180°/360°

扭动力稳定

反应快

SG90是一款9克级别的微型舵机、扭矩约为1.6kg·cm、工作电压4.8V至6V、可以通过50Hz的PWM信号控制其旋转角度（角度版本）或者旋转速度（360度连续旋转版本）。本设计使用的是180度的标准版本、通过控制舵机摆动一定的角度、将传送带上的包裹拨离原本的运动轨迹、进入到对应的分拣槽内即可。

SG90舵机的体积小、重量轻、价格便宜、对于本设计这种小型化的演示系统来说性能也完全够用。两个舵机分别安装在传送带的两个不同位置、分别对应目的地A和目的地B；目的地C则不需要舵机动作、包裹直接通过传送带的末端进入C筐就行了。这种“两个舵机表示三个目的地”的设计巧思不仅节约了硬件成本、也减少了机械结构的复杂度、整个系统看起来更加的简洁明了啊。

综上所述、本设计经过对功能需求、技术方案以及器件选型的多方面权衡之后、最终确定了以STM32F103C8T6为主控、MaixCAM为视觉前端、TB6612驱动直流电机带动传送带、SG90舵机执行分拣动作、OLED屏幕进行人机交互这样一个完整的系统方案。后续章节将围绕这个总体方案展开具体的硬件设计和软件实现工作。

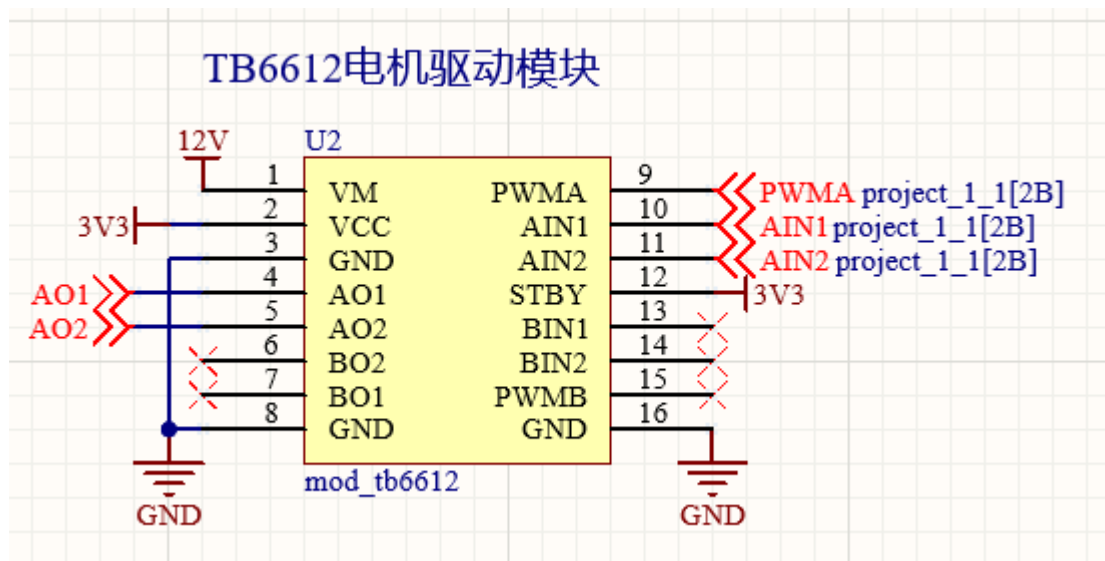
第三章 硬件设计

本章将围绕系统的硬件电路展开介绍。前面第二章已经完成了系统总体方案以及器件选型的工作，这一章将在此基础上从原理图的角度对各个功能模块的电路实现进行详细的说明。本人在硬件设计阶段使用了立创EDA作为原理图绘制工具，按照模块化的设计思想把整个电路拆分成了若干个独立的功能小块，每一块之间通过网络标号进行信号互联，这样做的好处是当某一部分需要调整时不会影响到其它的电路块，调试以及后续的修改都会方便许多。

下面将分别对TB6612电机驱动、舵机接口、摄像头接口、STM32最小系统以及复位电路这五个核心电路模块进行说明。

3.1 传送带电机驱动电路设计

传送带电机驱动电路是整个分拣系统中负责推动包裹运动的关键部分，本设计采用了基于TB6612FNG芯片的电机驱动模块来完成这一职能。具体的电路连接如图3-1所示。



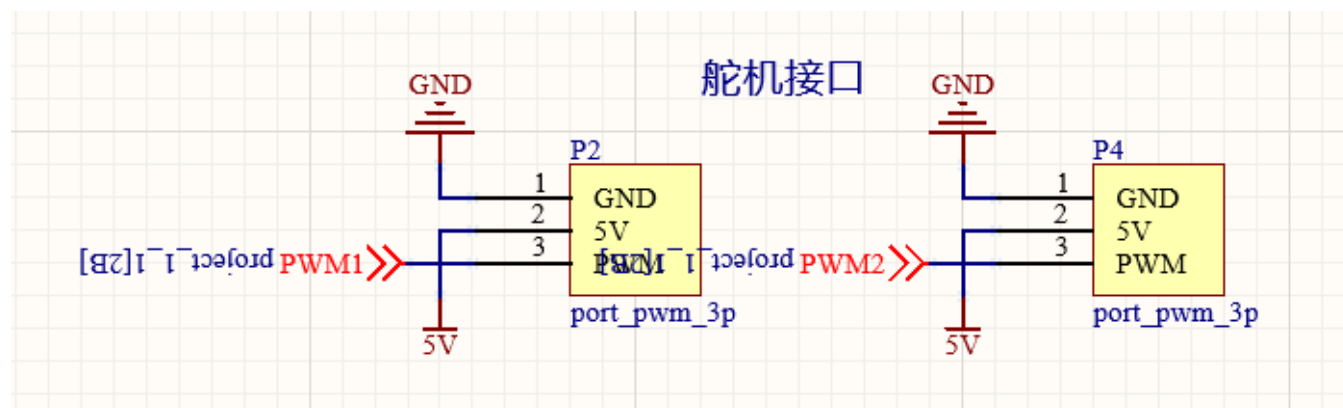
从图中可以看到，TB6612的1脚VM接的是12V电源，这一路12V由前级DCDC升压电路提供，专门用于给电机供电。2脚VCC接的是3.3V，这一路是给芯片内部的逻辑电路供电，与VM的12V是相互独立的两路电源，这种“逻辑电源与功率电源分离”的设计思路在大多数电机驱动IC上都是通用的做法，能够有效的隔离功率回路对逻辑回路的干扰。3脚和8脚是两个GND，需要并联起来共同接地，目的是降低地线的阻抗、提高大电流通过时地电位的稳定性啊。

芯片右侧的9、10、11脚分别是PWMA、AIN1、AIN2，构成了A通道电机的控制输入。本设计中只使用了A通道来驱动一颗直流电机带动传送带，其中PWMA接收来自STM32的PWM信号用于调节电机的转速，AIN1和AIN2则是两个方向控制位，通过这两个引脚的电平组合可以让电机正转、反转或者刹车。12脚STBY接的是3.3V，这是芯片的使能引脚，必须拉高才能让芯片正常工作，如果让它悬空或者下拉到地，整个芯片就处于待机状态、所有输出口都呈现高阻。13、14、15脚分别是BIN1、BIN2、PWMB，这一组对应的是B通道，本设计中没有使用因此把它们打了叉子标识为悬空。16脚是逻辑部分的地，与3脚、8脚共同接到系统地。

左侧的4、5、6、7脚是A0通道和B0通道的电机输出端口，4脚AO1和5脚AO2引出后接到外部接线端子上，再连接到传送带的直流电机上。本设计实测下来，TB6612在驱动这种小型直流电机时的发热非常的轻微，配合12V稳定电源给到VM端，电机的转速在一段长时间运行后能够保持非常的稳定，传送带的速度漂移控制在一个比较小的范围内，给后续的分拣时序计算带来了非常好的基础啊。

3.2 分拣舵机接口电路设计

舵机是本系统中负责执行分拣动作的关键部件，需要为它设计一个稳妥的接线接口。具体的接口电路如图3-2所示。



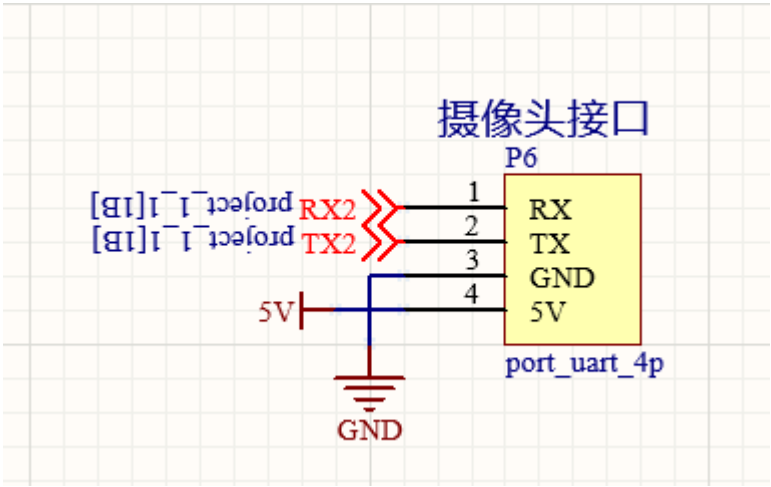
从图中可以看到，本设计为两个舵机分别预留了一个3pin的标准接口P2和P4，每一个接口都按照舵机行业的通用引脚定义进行布置：1脚为GND接地、2脚为5V电源正、3脚为PWM控制信号。这种3pin的舵机接口几乎是所有市面上舵机的统一标准，使用通用的杜邦线就可以直接进行连接，方便了实验阶段的快速搭建以及后续可能的舵机替换啊。

需要特别说明的是，舵机的电源供电5V这里使用的是 PWR_5V 这条专用电源轨，而不是与单片机共用的那一路5V。前面第二章中本人已经分析过，舵机属于感性负载，启动以及换向的瞬间会产生比较大的电流尖峰，这种尖峰如果耦合到单片机所在的5V电源轨上，就有可能造成单片机的工作电压瞬间下跌，进而导致复位甚至程序跑飞的现象发生。所以在原理图设计阶段就把舵机的5V与其它模块的5V做了物理隔离，分别引出两条独立的电源走线，从源头上规避掉了这种串扰风险。

PWM1和PWM2分别来自STM32的两个定时器通道，频率统一设置为50Hz、占空比通过软件动态调节。SG90舵机的标准控制协议是：占空比1ms对应0度位置、1.5ms对应90度中位、2ms对应180度位置，本设计通过这种PWM占空比的精确调节就可以让舵机准确的转动到期望的角度，从而把传送带上的包裹推到对应的分拣口里去。

3.3 摄像头串口通信接口电路设计

摄像头模块与主控之间通过串口进行通信，为了实现物理层的连接，本设计设计了一个4pin的UART接口，原理图如图3-3所示。



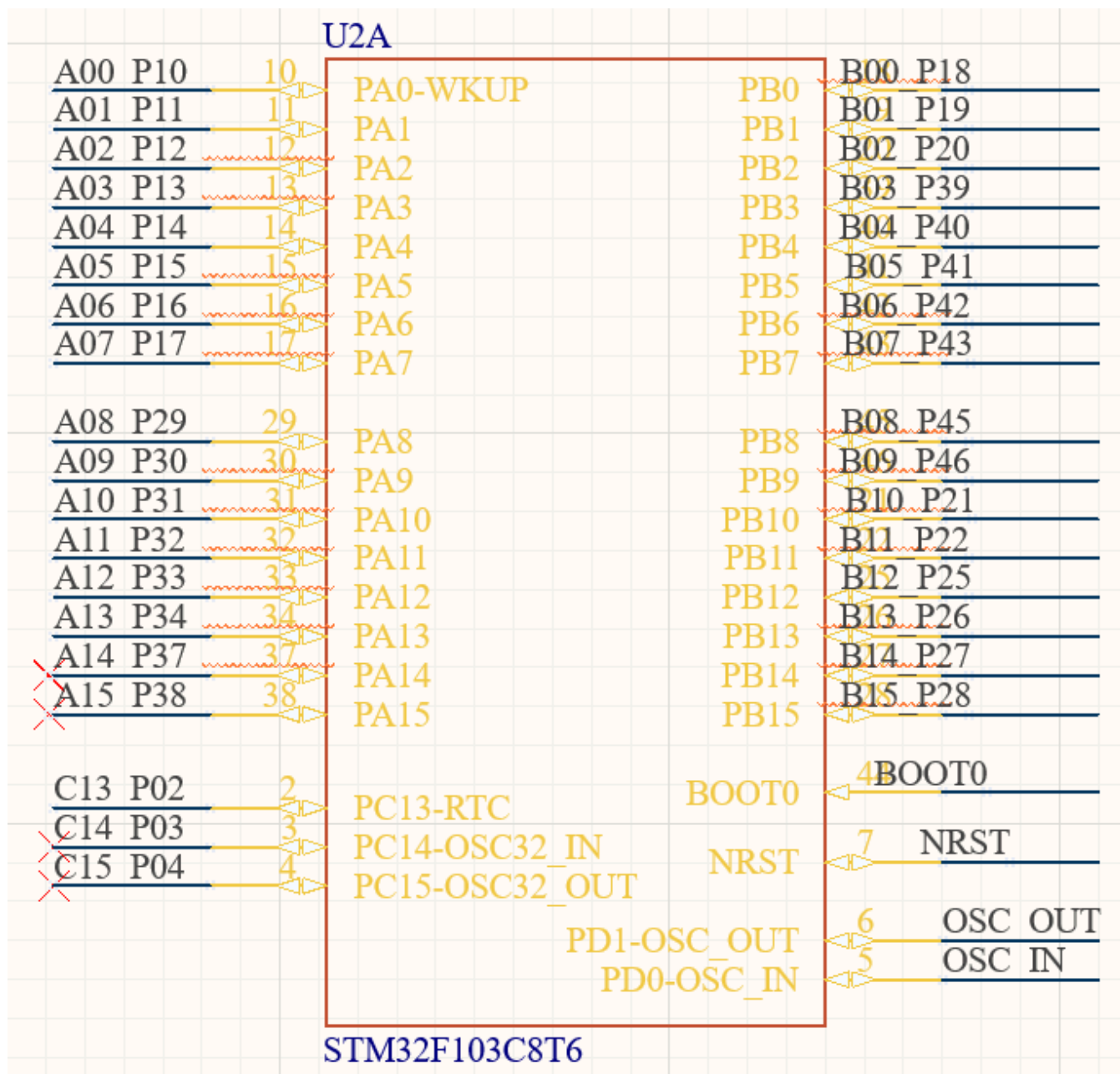
接口P6是一个4pin的标准UART接线座，引脚定义从1到4分别为RX、TX、GND、5V。这种排列顺序是参考了大多数串口模块的通用约定，使用普通的4芯杜邦线就可以与摄像头模块直接对接，省去了自制连接线的麻烦。其中5V这一路用于给MaixCAM摄像头模块提供工作电源，GND是参考地，RX和TX则是数据收发的两根信号线啊。

需要注意的是，串口通信中RX和TX是要交叉连接的，也就是说STM32端的TX要接到摄像头端的RX、STM32端的RX要接到摄像头端的TX，这样双方才能正确的进行半双工或全双工的数据交换。在原理图上，本人把这两个网络命名为RX2和TX2，这两个网络分别对应STM32的USART2外设的接收和发送引脚。USART2运行在115200波特率，8位数据位、1位停止位、无校验，这是一组比较通用的串口配置，足以满足摄像头模块每秒发送十几帧数据的吞吐需求啊。

为什么不使用USART1呢，主要是因为USART1已经被预留给了将来可能扩展的调试串口或者无线通信模块使用，把摄像头放在USART2上可以让两个串口的功能各司其职、互不干扰。

3.4 STM32主控最小系统电路设计

STM32F103C8T6作为整个系统的"大脑"，其最小系统电路是其它所有模块得以正常工作的前提。最小系统的引脚分布如图3-4所示。



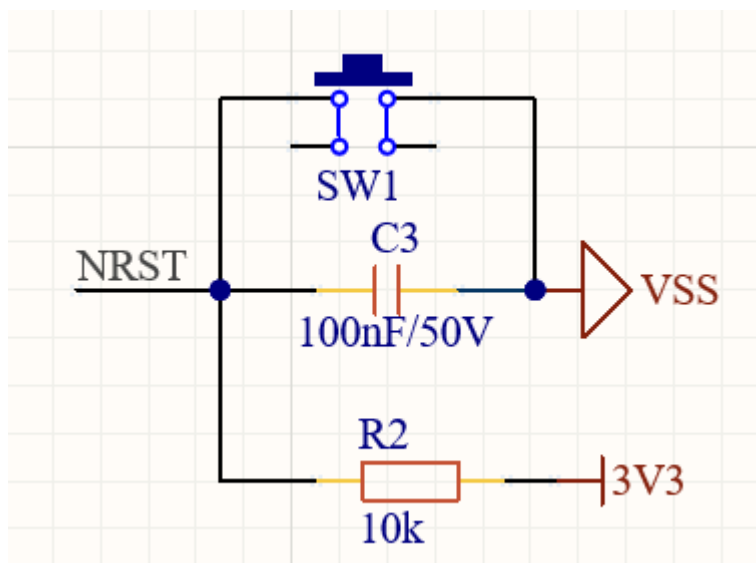
从图中可以看出，本设计完整的引出了STM32F103C8T6的所有可用GPIO引脚，包括A组的PA0至PA15、B组的PB0至PB15以及C组的PC13至PC15，每一个引脚都通过网络标号引出到外部的功能模块。其中PA14和PA15被打上了叉子标识，是因为这两个引脚默认是作为SWD调试接口使用的（PA13是SWDIO、PA14是SWCLK），如果占用了这两个脚就没办法通过ST-Link进行在线烧录与调试，所以本设计选择把它们保留给调试器使用、不挪做它用啊。

PC14和PC15默认是连接到外部低速晶振（OSC32_IN和OSC32_OUT）用的引脚，但是本设计没有使用RTC功能，所以这两个引脚也被空闲了出来；PD0和PD1则是连接外部高速晶振（OSC_IN和OSC_OUT）的引脚，连接到一颗8MHz的晶振电路上，配合STM32内部的PLL可以倍频到72MHz作为系统主频。

BOOT0引脚通过下拉到GND使得芯片每次上电后从主Flash启动用户程序；NRST是芯片的复位引脚，连接到外部的复位电路（详见3.5节）。整体看下来这个最小系统的设计是比较紧凑且功能完整的，能够支撑起本系统所需的所有外设接口啊。

3.5 系统复位电路设计

复位电路虽然看似简单，但却是嵌入式系统中不可或缺的一部分，其原理图如图3-5所示。



复位电路由一颗按键SW1、一个100nF的去耦电容C3以及一颗10kΩ的上拉电阻R2组成，构成了一个非常经典的RC复位结构。在没有按下按键的常态下，3.3V电源通过R2上拉电阻把NRST引脚拉到高电平、芯片正常运行；当按键被按下时，NRST被直接短接到地、电平变低，芯片进入复位状态；按键释放后，电源通过R2再次给C3充电，NRST电平随着C3的充电过程缓慢回到高电平，芯片重新启动从复位向量开始执行程序。

C3这个100nF电容的作用有两个，第一是滤除NRST引脚上可能感应到的高频干扰噪声，避免在没有按键动作的情况下出现意外复位；第二是配合R2构成RC时间常数大约为1ms的延时网络，使得复位释放时NRST的电平变化平缓，给芯片内部的复位电路足够的稳定时间啊。这种设计在STM32官方的参考设计文档当中也是被强烈推荐的标准做法，本设计直接沿用了这一经典电路。

第四章 软件设计

软件设计是整个系统的核心部分，硬件电路只是搭建了一个能够运行的物理平台、真正赋予系统“智能”的还是软件。本章将围绕下位机也就是STM32端的软件展开详细介绍，从操作系统的选择、外设的配置一直讲到具体的功能实现。

4.1 下位机软件设计

4.1.1 FreeRTOS 操作系统

在嵌入式开发当中，是否使用操作系统这个问题需要根据具体的项目需求来权衡。如果只是简单的轮询执行几个动作，那么用裸机代码加while大循环可能就足够了；但是当系统中存在多个具有不同实时性要求的任务时，引入一个RTOS就能够让代码的结构更加的清晰、调度更加的有序。本设计的工作流程涉及到了串口数据接收、PWM输出、I2C通信、按键扫描以及包裹时序管理等多个并行的功能模块，这些功能各自有不同的执行周期和优先级要求，因此本人最终决定引入FreeRTOS这一款被广泛应用的开源实时操作系统作为软件的运行基础。

FreeRTOS本身是一个开源、可裁剪、易移植的实时内核，最大的特点就是足够的轻量化，整个内核仅占用十几KB的Flash和几KB的RAM，对STM32F103C8T6这种资源比较有限的MCU来说也是完全可以承受的。本设计通过CubeMX图形化工具引入了CMSIS_V2接口的FreeRTOS，配置了两个不同周期的任务来分别处理系统级与应用级的逻辑。

经过分析与权衡之后，本人将系统的软件代码组织为应用层、驱动层和基础库三层结构。应用层负责具体的业务逻辑实现，驱动层封装各个外设的操作接口，基础库提供通用的工具函数比如延时、CRC校验、字符串处理等。这种分层设计的好处是上层代码不需要关心底层硬件的具体实现细节，只需要调用驱动层暴露出来的标准接口即可，将来如果要把这套代码移植到别的MCU平台上去也会比较的方便。

具体的任务规划如表4-1所示。

任务名	调度周期	主要职责
systemTask	100ms	维护基础库的状态机以及通信更新
user1Task	2ms	包裹识别、电机控制、舵机控制等核心业务逻辑

可以看到本设计共创建了两个 FreeRTOS 任务，分别负责系统级的周期事务以及应用级的实时业务。其中user1Task 是主要的工作任务，它以2ms为基础调度周期循环运行，在每一次循环中通过对内部计数器taskCnt取余的方式实现 10ms、100ms、500ms等多级时间片的调度。这种"基础周期+计数取余"的设计巧思使得本人可以在同一个任务里面优雅的安排多个不同周期的子任务，而不需要为每一种周期都单独开一个RTOS任务、节省了系统资源也降低了任务间同步的复杂度啊。比如说核心控制循环以2ms周期运行、通信收发以10ms周期运行、OLED屏幕刷新以100ms周期运行，每一类逻辑都安排在最合适的时间片上执行。

4.1.2 STM32 基础外设配置

为了让系统能够正常工作，需要在程序启动的早期阶段对各个外设进行初始化配置。本设计利用CubeMX工具对 STM32F103C8T6的外设进行了图形化的配置，主要使用到的外设包括了GPIO、TIM、USART以及I2C四类，下面分别进行说明。

GPIO方面，主要用到了几个普通输入输出口，比如按键输入、外部状态指示等，这些引脚均按照3.3V CMOS电平标准进行配置、输入端打开内部上拉、输出端则配置为推挽输出模式。

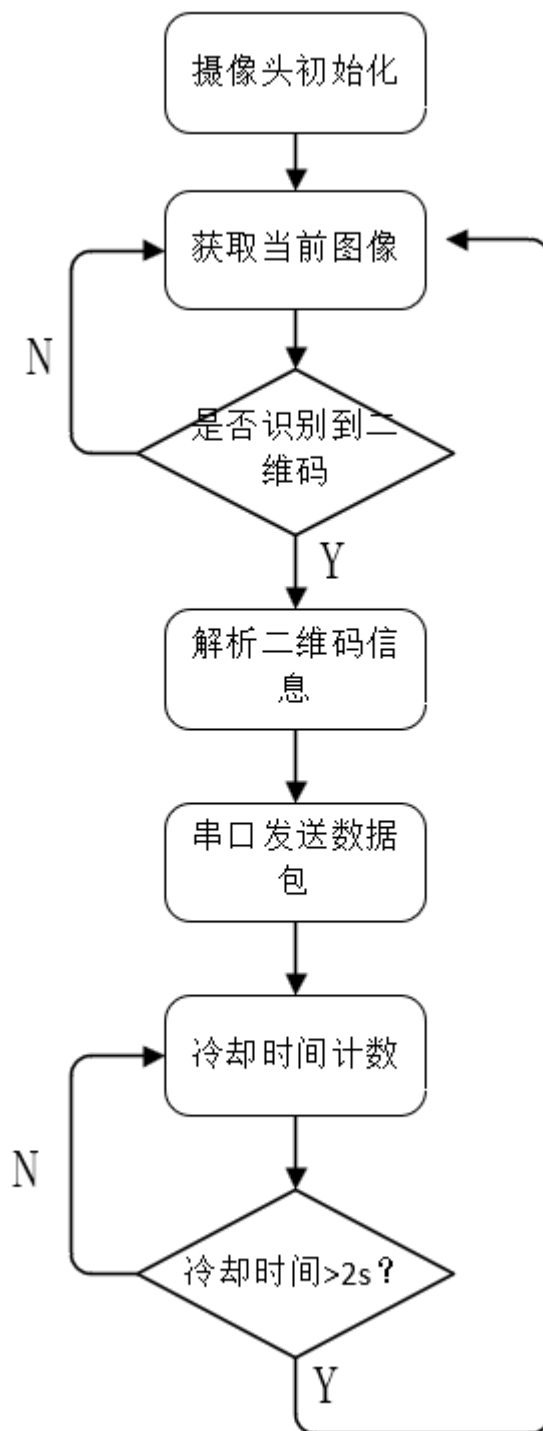
TIM定时器方面，本设计使用了TIM2、TIM3、TIM4三个定时器，分别用于产生不同的PWM信号。其中TIM2用于驱动 TB6612电机驱动模块的PWMA输入端、控制传送带电机的转速；TIM3和TIM4则分别用于驱动两个SG90舵机、控制它们的旋转角度。所有定时器的预分频系数与重装载值都经过了精心的计算，使得电机PWM频率为10kHz左右，舵机 PWM频率统一为50Hz、与SG90的标准控制协议相符合。

USART方面，本设计使用了USART2来接收来自MaixCAM摄像头模块的数据。波特率设置为115200bps，8位数据、1位停止位、无校验位、无硬件流控，这是一组兼容性很好的标准串口配置。为了避免在主循环中阻塞等待数据，串口接收使用了DMA加空闲中断的方式，DMA负责把每一个字节自动搬运到环形缓冲区中、空闲中断在一帧数据接收完毕后通知应用层进行解析处理。

I2C方面，使用了I2C1接到OLED显示屏上、采用了软件模拟I2C的方式以提高灵活性。OLED的I2C地址默认是0x78，通过SCL和SDA两根线就可以完成所有命令以及数据的传输啊。

4.1.3 摄像头二维码识别功能软件设计

摄像头侧的二维码识别功能由Sipeed MaixCAM模块独立完成，主控STM32只需要被动接收摄像头发过来的串口数据帧、解析出其中的目的地编码即可。摄像头端的Python识别程序流程如图4-1所示。



从流程图中可以看到，整个识别过程被设计为一个简单的有限状态机，包含了IDLE（空闲）、SENT（已发送）和COOLDOWN（冷却）三个状态。系统启动以后摄像头默认处于IDLE状态，持续不断的从图像传感器读取每一帧画面、调用 find_qrcodes 函数尝试识别二维码。当成功识别到二维码以后，状态机会立即跳转到SENT状态、把识别出来的目的地信息封装成16字节的固定格式数据帧通过串口发送给STM32。

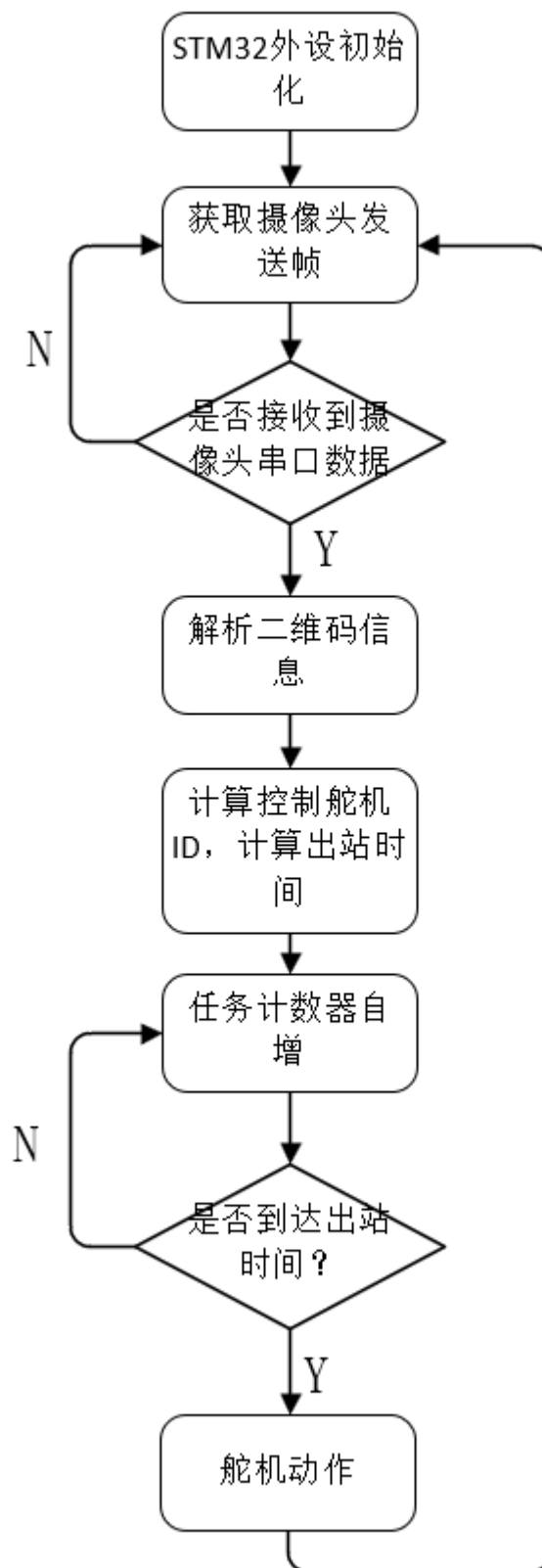
数据帧的具体格式是这样的：第一个字节是包头标识0x55、第二个字节是包头标识0xAA，这两个字节组合起来的0x55AA是一个比较有辨识度的同步字节序列，可以让接收端容易的找到一帧数据的起点；第三个字节是控制字段固定为0x01；第四到第十三个字节是真正的二维码内容、最长10字节，不足的部分用0x00填充；第十四个字节是CRC8校验值，多项式选用了0x07这一标准CCITT-CRC8的多项式；第十五个字节是包计数器，每发送一帧自增一次方便接收端检测丢包；最后一个字节是包尾0xFF用于标识一帧数据的结束。

数据发送完毕后，状态机进入COOLDOWN冷却状态，这个状态会持续2000ms。在冷却状态期间，即使摄像头继续看到同一个二维码也不会再次发送数据帧，这样做是为了防止同一个包裹由于在摄像头视野下停留较长时间而被重复识别多次、给后续的分拣逻辑造成混乱。冷却时间结束以后状态机才会重新回到IDLE状态、准备识别下一个新的包裹啊。

需要特别提一下的是CRC8校验机制。本人在代码中通过 ENABLE_CRC 全局变量提供了一个开关，可以方便的在调试阶段关闭校验、在正式运行阶段再打开。CRC8的计算函数使用经典的逐位异或加左移结构，每一个输入字节先与CRC寄存器异或、然后循环8次、每次根据最高位是否为1决定是否与多项式异或，这是CRC算法的标准实现，运行起来非常的高效。

4.1.4 STM32 控制核心功能软件设计

STM32端的控制核心是整个系统的"大脑"，负责接收摄像头的识别结果、维护包裹列表、计算分拣时序以及驱动舵机执行动作。其总体的程序流程如图4-2所示。



从流程图中可以看出，STM32端的工作流程也可以理解为一个无限循环的状态轮询过程：系统上电后首先完成各种外设的初始化，然后进入主循环。在每一个循环周期内，程序会依次去查询是否有摄像头发来的新数据、解析出二维码内容并计算出该包裹的预期出站时间、然后让任务计数器自增、判断列表中是否有包裹到了出站时间、如果到了就驱动相应的舵机做出动作。

下面结合代码来进一步说明这套逻辑的具体实现。在主应用任务的更新函数中，首先是依次调用了几个关键的子函数：

```
DRIVER_BOARD_KeyInfoUpdate();
sysControl();
motorControl();
servoControl();
DRIVER_VERISON_GetMaixCamInfo();
packRecognize();
```

这几个函数构成了主应用任务的核心调度脉络。其中 DRIVER_BOARD_KeyInfoUpdate 用于更新按键状态，本设计中按键KEY1是用来启动整个分拣过程的、只有当用户按下KEY1（或者通过手机APP远程控制）系统才会真正开始运行、否则保持待机以方便人工放料。

包裹识别的核心逻辑封装在 packRecognize 函数中。当摄像头模块发来一帧新的数据后，驱动层会置位一个 k210RxFlag标志，应用层在每一次循环中通过 DRIVER_VERISON_HasNewData 函数来检查这一标志，如果为1则说明有新数据到来。新数据到来后，函数根据其中的目的地编码（var1字段）通过switch语句进行分支处理：

```
switch(versionRealData.field.var1){
    case 0x31: /* 目的地A: 延迟时间后触发舵机1 */
        packList[sysCtrl.packNum].outportTime = sysCtrl.taskCnt + CONVEYOR_DELAY_A;
        packList[sysCtrl.packNum].servoId = STEER_SERVO_1;
        break;
    case 0x32: /* 目的地B: 延迟时间后触发舵机2 */
        packList[sysCtrl.packNum].outportTime = sysCtrl.taskCnt + CONVEYOR_DELAY_B;
        packList[sysCtrl.packNum].servoId = STEER_SERVO_2;
        break;
    default: /* 目的地C: 不触发任何舵机、直接通过 */
        packList[sysCtrl.packNum].outportTime = 0U;
        packList[sysCtrl.packNum].servoId = SERVO_ID_NONE;
        break;
}
```

可以看到，本人为目的地A的包裹安排了 CONVEYOR_DELAY_A 这个固定的时间延迟之后才触发舵机1、为目的地B的包裹安排了 CONVEYOR_DELAY_B 之后触发舵机2，而对于目的地C的包裹则不安排任何动作、让它顺着传送带自然的滑到最末端的C筐里。这两个延迟值是通过实际测量得出的——在传送带速度恒定的前提下，包裹从摄像头扫码点到舵机1处需要 CONVEYOR_DELAY_A 个调度周期、到舵机2处需要 CONVEYOR_DELAY_B 个调度周期。

每一个新识别到的包裹被添加到 packList 这个数组当中、保存其目的地、出站时间、舵机ID等信息。因为应用任务以 2ms周期运行，所以taskCnt每自增1代表过去了2ms，整个时序的计算精度可以达到毫秒级别。

紧接着是 sysControl 函数，这个函数会遍历整个 packList 列表、检查每一个未处理的包裹是否到了它预定的出站时间。判断的方式是用当前taskCnt减去包裹的outportTime、如果差值大于等于0就说明到了或者已经超过了触发时刻、应当立即执行舵机动作。这里需要特别留意的一个点是本人采用了 (int32_t) 强制类型转换的写法：

```
(int32_t)(sysCtrl.taskCnt - packList[i].outportTime) >= 0
```

之所以这样写而不是直接用无符号比较，是为了规避32位计数器溢出回绕带来的问题——当taskCnt接近溢出回到0之后，无符号比较会得出错误的结果，而强制转换为有符号数后差值会自然的变成一个很小的负数、保持判断的正确性。这是一种处理“循环计数器”的常见技巧，也是本人在实际调试中踩坑后总结出来的一个经验啊。

电机控制的逻辑封装在 motorControl 函数中。这个函数实现了一个简单的速度斜坡启动机制：每次调用 motorSpeed 都会增加一个 MOTOR_SPEED_RAMP_STEP 步长，直到达到 TB6612_SPEED_MAX 最大速度后保持稳定。这种斜坡启动可以避免电机启动瞬间的电流冲击、保护电机以及电源、同时也让传送带的启动过程更加的平滑：

```
if(motorSpeed < TB6612_SPEED_MAX){
    motorSpeed += MOTOR_SPEED_RAMP_STEP;
} else {
    motorSpeed = (int16_t)TB6612_SPEED_MAX;
}
DRIVER_TB6612_MotorSetSpeed(TB6612_MOTOR_A, -motorSpeed);
```

调用 DRIVER_TB6612_MotorSetSpeed 函数时传入了 -motorSpeed 也就是带负号的速度值，这是为了让电机按照预期的方向旋转——根据实际的硬件接线情况，本设计中传送带需要电机反转才能让物块从摄像头一端运动到舵机一端。

舵机控制的 servoControl 函数则负责把 sysControl 函数标记的待动作的舵机真正的执行旋转。本设计使用的是360度连续旋转改装的SG90舵机，通过 DRIVER_STEER_Rotate360 函数控制其旋转方向以及持续时间：

```
DRIVER_STEER_Rotate360(STEER_SERVO_1, STEER_DIR_CW, 400); /* 顺时针旋转、拨开包裹 */
DRIVER_STEER_Rotate360(STEER_SERVO_1, STEER_DIR_CCW, 390); /* 逆时针旋转、回到原位 */
```

可以看到每一次完整的分拣动作由两步构成：先以顺时针方向旋转一段时间把包裹拨离传送带、再以逆时针方向旋转一段相近的时间让舵机臂回到原始位置等待下一次动作。两个时间值400和390不完全相等是因为机械上存在一定的回程间隙、需要稍微调整以保证舵机能够准确回位。两个舵机的旋转方向恰好相反（舵机1是先CW后CCW、舵机2是先CCW后CW），这也是因为它们安装在传送带的不同侧、需要朝相反的方向把包裹推开。

4.1.5 OLED显示模块软件设计

OLED显示屏在本系统中扮演了人机交互窗口的角色，负责实时的把当前的系统状态信息呈现给操作者，方便调试以及运行时的状态监控。具体的显示内容由 oledControl 函数实现：

```
DRIVER_OLED_ShowString(5,5," PACKAGE SORT SYSTEM ");
DRIVER_OLED_ShowString(5,15,"system time:");
DRIVER_OLED_ShowNum(80, 15, systemTaskInfo.systemTaskCnt/10, 3);
DRIVER_OLED_DrawPoint(100,20,1);
DRIVER_OLED_ShowNum(102, 15, systemTaskInfo.systemTaskCnt%10, 1);
DRIVER_OLED_ShowString(110,15,"s");
```

这一段代码的功能是在OLED屏幕上显示系统运行时间、精确到0.1秒。systemTaskCnt是 systemTask 这个100ms周期任务的计数器，所以systemTaskCnt/10就是已运行的整数秒数、systemTaskCnt%10则是当前秒内的0.1秒部分，两者中间用一个小数点连接起来构成了一个完整的"秒.十分之一秒"格式的时间显示啊。

除了系统时间以外，OLED还会显示已扫描到的包裹总数 maixCamInfo.rxTotalCnt 以及当前的系统使能状态key1，让用户能够一目了然的了解到整个分拣过程的进度。考虑到I2C协议的传输速度相对比较慢、频繁刷新会拖累整个控制循环的实时性，本人特意把OLED的刷新频率限制在了每100ms一次、也就是10Hz的刷新率。这个频率对人眼来说已经足够平滑、看不出任何卡顿、但对MCU来说就轻松了很多：

```
if(sysCtrl.taskCnt % 50u == 0u){
    oledControl();
}
```

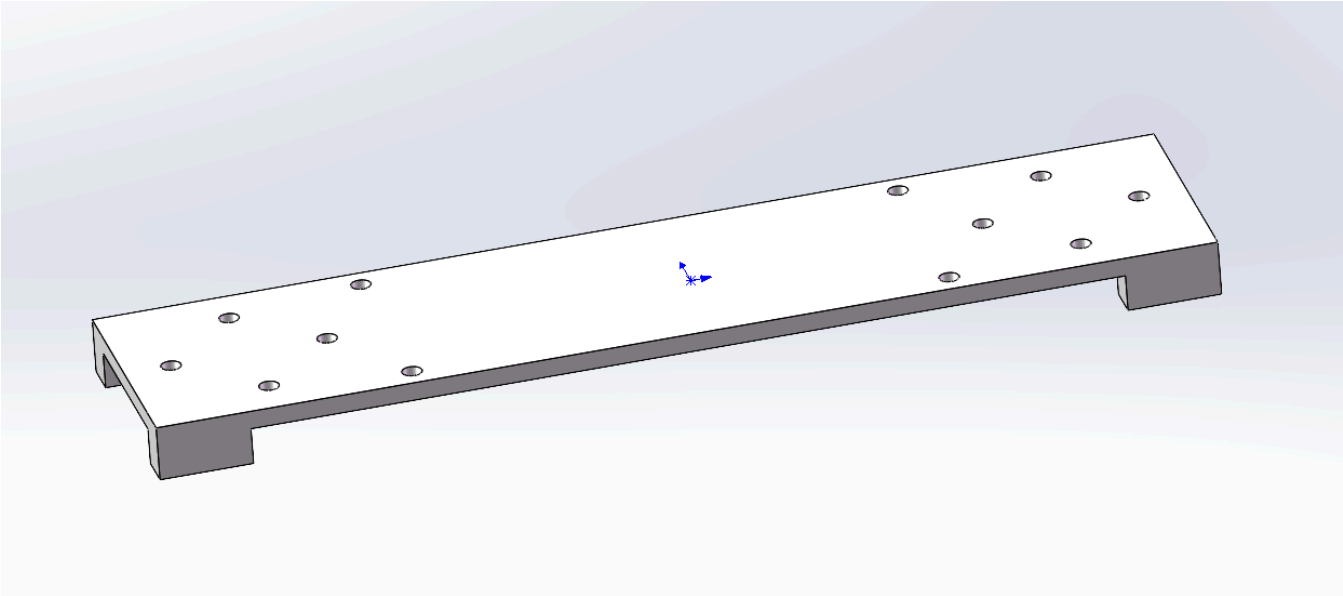

这里taskCnt每2ms自增一次、所以对50取余就相当于每100ms执行一次oledControl函数、刚好达到我们想要的刷新频率。

第五章 机械外观设计

除了电路与软件之外，机械结构也是本系统中不可或缺的组成部分。良好的机械结构能够把分散的电子元件有机的整合到一起、让整个系统看起来像一个完整的产品而不是一堆松散的零部件堆砌。本人在SolidWorks软件中完成了所有结构件的三维建模工作、然后通过3D打印技术加工出实物。下面将分别介绍几个主要的结构零件。

5.1 舵机固定底板设计

舵机固定底板是整个机械结构的基础，承担了支撑两个分拣舵机以及连接其它附属机构的功能。具体的三维模型如图5-1所示。

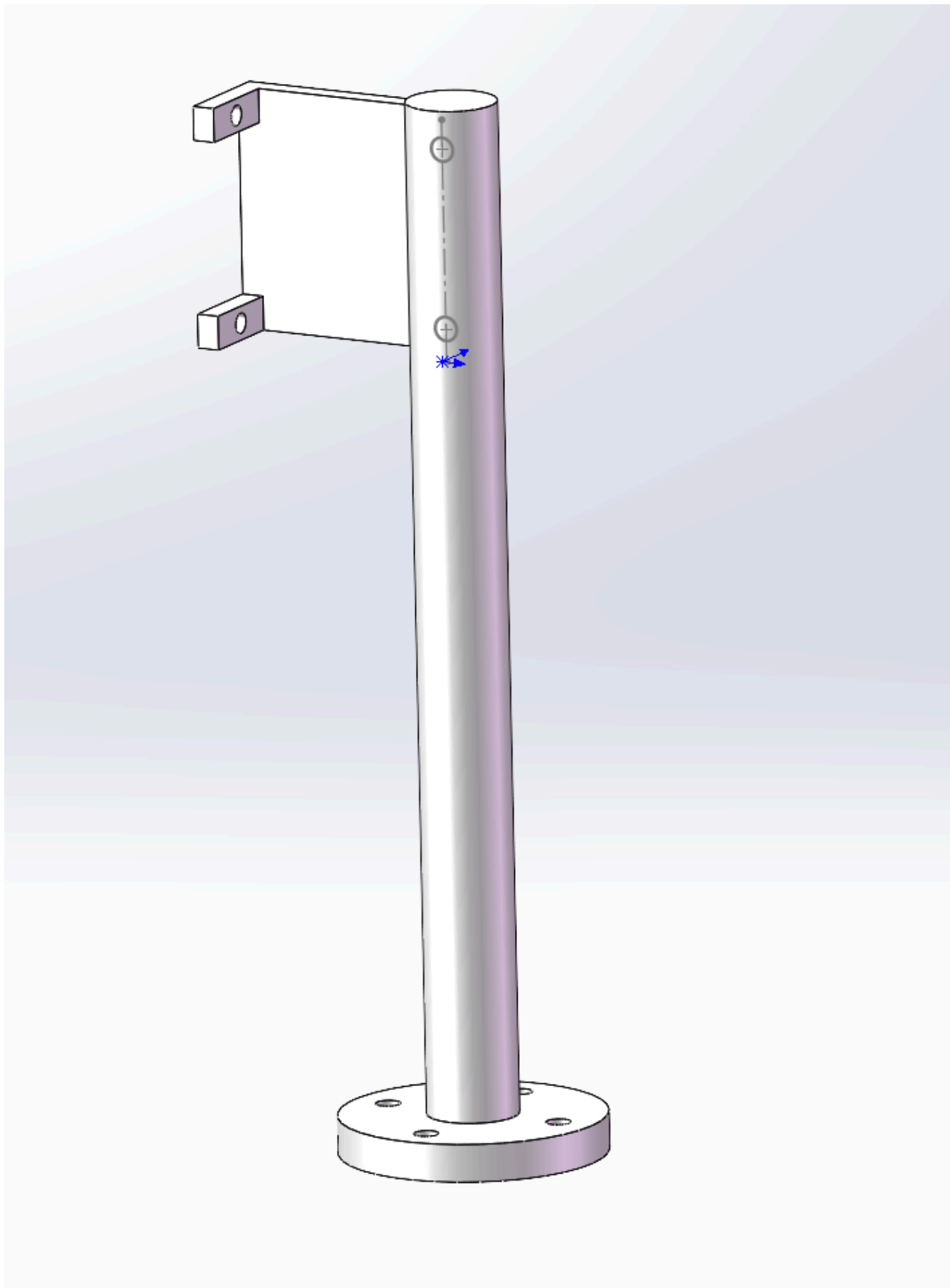


从图中可以看出，底板的整体形状是一个长条形的板状结构，板面上分布有多组规整排列的圆孔，这些孔位的设计经过了精心的考量。靠近板的两端各有几个较大的圆孔，是用于通过M3螺钉把底板固定到下方的支架立柱上去；中间区域的小孔则是用于安装两个SG90舵机的固定螺钉。SG90舵机的标准安装方式是通过其外壳两侧的两个穿孔螺丝固定，本设计的孔位间距严格按照SG90的官方尺寸（27.8mm）进行了对齐、保证舵机安装后没有任何的松动。

底板的两端略微向下凹陷形成了一对支脚结构，这样的设计是为了在不需要额外加强筋的情况下增强底板的整体刚度、防止其在舵机启停产生的反作用力下发生形变。整个底板的材料选用的是PLA塑料、通过3D打印一体成型，重量轻、强度也足够、加工成本也比较的低，对于本毕业设计这种小批量原型来说是非常合适的选择啊。

5.2 舵机固定支架设计

舵机固定支架的作用是把5.1节中的固定底板架空到适合的高度、使得安装在底板上的舵机能够刚好作用到传送带上方的包裹。其具体的造型如图5-2所示。



从图中可以看到，这个支架的整体造型由两部分组成：上部是一个U形夹片用于和底板进行装配连接、下部是一根圆柱形的立柱用于把整体撑高。U形夹片的两个臂上各开了一个圆孔、与5.1节底板上对应的孔位匹配、通过两颗M3螺钉穿过后用螺母锁紧、就可以把底板牢固的悬挂在立柱顶端。

立柱采用了实心圆柱的设计、直径选取的是14mm左右、这个粗细既能保证足够的抗弯刚度、又不会显得过于笨重。立柱的高度是根据传送带的实际高度量身定做的、本设计中支架立柱的总长度大约是180mm左右、刚好让上方的舵机臂能够伸到传送带上方约10mm的位置、既不会碰到传送带本身、又能在转动时把包裹有效的拨开。整个支架同样采用PLA材料3D打印成型、为了提高强度本人把打印的填充率从默认的15%提高到了40%、确保在受到舵机动作时反作用力的情况下不会出现折断的危险。圆形的法兰底座。法兰盘上均匀的分布有4个螺钉过孔、可以通过M4螺钉把整个支架牢固的固定在桌面、铝型材或者实验板上、避免在使用过程中发生倾倒。法兰盘的直径设计为60mm、相比于立柱本身的直径有了显著的扩大、这种"上窄下宽"的造型从力学上讲能够提供非常好的抗倾覆能力。

把这种带底座的支架与5.2节的简易支架混合使用,就可以构建出一个稳定的舵机分拣台架——简易支架用在结构内部不方便安装底座的位置、带底座支架则用在外围承担主要的固定职能。整个机械结构在装配完成以后非常的牢固、即使长时间持续运行舵机动作也不会出现任何的松动或位移、为整个分拣系统提供了一个非常可靠的物理基础啊。

至此本章对系统的硬件电路、软件实现以及机械结构这三个方面都进行了相对详细的介绍。下一章将围绕系统搭建完成后的实际测试展开论述、通过具体的测试数据来验证本设计是否达到了任务书所规定的各项指标要求。

第六章 系统测试

系统软硬件搭建完成后,笔者必须对整机进行一轮完整的测试,以验证各功能模块是否达到任务书所要求的指标。本章我按照从底层到上层的顺序展开,先验证电源支路的稳定性,再验证电机驱动的输出波形,接着验证摄像头到主控之间的串口通信链路,最后给出二维码素材的制作环节以及整机联调的结果。本章所采用的测试设备主要为普源 RIGOL DH0924 数字示波器,该示波器具备 1.25GSa 每秒的采样率和 250MHz 的带宽,完全够用于本系统中各种低速信号的捕获与协议解码。每一项测试我都会先说测试方法,再讲测试现象,最后给出结论,确保测试过程可以被复现。

6.1 板载 5V LDO 电源稳压性能测试

电源是整个系统的根基,如果电源纹波过大或者输出电压偏离额定值,后续所有模块都会出现各种各样的怪问题。所以笔者把电源测试放在第一位来做。本节测试的是分压电路输出的 5V LDO 支路,这一路 5V 主要给 STM32 最小系统板和摄像头模块提供工作电压,要求输出稳定,纹波尽量小。

测试方法很简单啊,就是把示波器探头钳的地夹在分压电路的 GND 测试点上,探头尖触在 5V LDO 的输出焊盘,然后让整机正常上电运行,在传送带电机和舵机都处于工作状态下采集波形。这种工况是最严酷的,因为电机在转的时候会通过共地耦合产生噪声干扰电源支路。我把示波器的垂直档位拉到 100mV 每格,采用直流耦合,这样既能看到 5V 的直流分量,又能观察到叠加在直流上的纹波。



测试现象如图 6-1 所示,示波器读数显示通道 1 的直流值稳定在 4.98V 附近,与目标 5V 的偏差在 20mV 以内,这个偏差完全可以接受。波形整体表现为一条平直的直线,只在某些时刻出现非常细微的尖峰,峰峰值我目测大约在 30mV 左右,从波形上看这些尖峰呈现出周期性的特征,我推测是舵机 PWM 切换边沿通过电源耦合过来的开关噪声。由于尖峰幅度很小,远远小于 STM32 和摄像头的供电容差范围,所以这个 LDO 的稳压效果是符合设计要求的。结论上来说,5V LDO 这一路完全能够支撑前端逻辑电路的稳定运行,本次测试通过。

值得说明的是,我之所以选择 LDO 而不是 DCDC 来给逻辑电路供电,正是出于纹波考虑。LDO 虽然效率不如 DCDC,但是它的输出更干净,对于带摄像头这种对电源敏感的负载来说,几乎等于零纹波的输出特性是非常关键的。当然这一路只带 STM32 和摄像头,负载电流不大,LDO 的发热问题也可以忽略不计。

6.2 STM32 板载 3.3V LDO 稳压输出测试

STM32F103C8T6 工作在 3.3V 电压域,这个 3.3V 是由最小系统板上自带的 LDO 从外部输入的 5V 降压得到的。这一路电压直接关系到主控芯片能否稳定运行,关系到 GPIO 电平是否标准,所以单独再做一次测试。

测试方法跟上一节几乎相同,我把示波器探头接在 STM32 最小系统板上 3V3 输出排针上,地线接在板子的 GND 排针上,垂直档位调到 100mV 每格,水平档位调到 100us 每格,采集 STM32 在执行业务逻辑时的供电波形。这个时候 STM32 内部 USART 中断在跑,定时器在产生 PWM,I2C 在跟 OLED 通信,负载是动态变化的,正好能反应出真实的稳压性能。



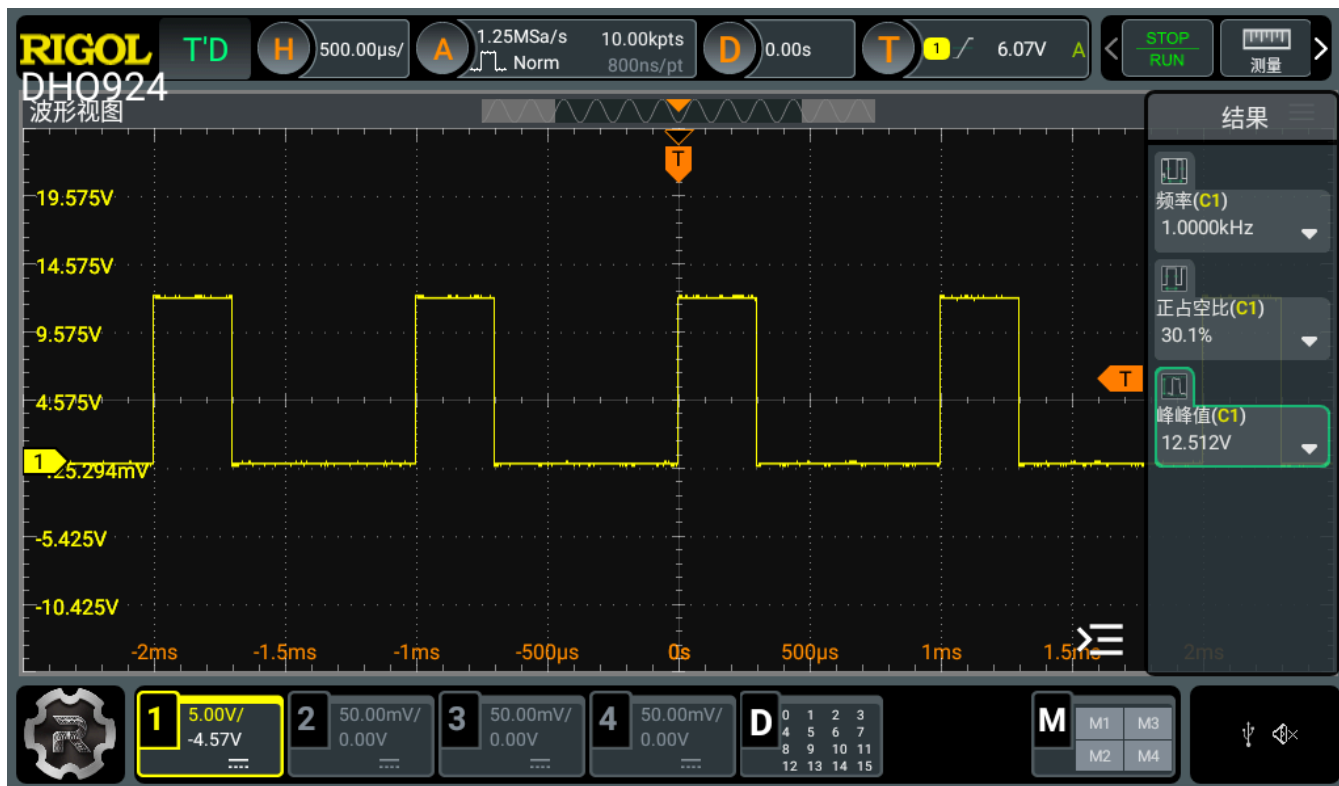
测试结果如图 6-2 所示,通道 1 直流读数为 3.35V,与标称的 3.3V 几乎重合,偏差仅 50mV,在 LDO 的精度范围内。波形上能观察到一系列细密的小尖刺,峰峰值大约 80mV 左右,密度比 5V 那一路要高一些。我分析这是因为 STM32 内部数字逻辑切换非常频繁,每个时钟周期都有大量的开关事件,这些电流波动通过电源回路反映在输出端,就形成了这种密集的高频噪声。但是这些小尖刺的幅度依然没有超过 STM32 的供电容差范围 3.0V 到 3.6V,芯片在测试期间没有出现任何复位或者死机现象,因此 3.3V 这一路的稳压性能是合格的。

笔者在调试初期曾经遇到过 STM32 莫名其妙复位的情况,后来才发现是因为锂电池的电压在大电流冲击下跌落,经过 LDO 后导致 3.3V 也跟着掉,触发了欠压复位的现象出现。后来我把电池容量加大,并且在 LDO 的输入端并联了一个 470uF 的钽电容用来储能,这个问题就再也没有出现过哇。所以电源设计这一块,光是输出端的设计还不够,输入端的能量缓冲也很重要。

6.3 TB6612 电机驱动 PWM 波形测试

电机驱动的核心是 PWM 波形的质量,如果占空比不准、频率漂移、上升下降沿不干净,都会导致电机运行不平稳,从而影响传送带的速度恒定性。本节测试的目标是验证 STM32 输出给 TB6612 的 PWM 信号是否符合软件设定值,以及 TB6612 经过 H 桥放大后输出给电机的电压波形是否正确。

测试方法上,笔者把 TB6612 的电机输出端 AO1 或者 AO2 接到示波器通道 1,GND 公共参考接示波器的接地夹。软件中我把电机调速值设置为 30 个百分点的占空比,这是一个比较温和的工作点,能让传送带以一个适中的速度运转,既能保证物块在传送带上的稳定移动,又能让摄像头有足够的时间识别二维码。示波器的水平档位调到 500us 每格,这样能够看到几个完整的 PWM 周期,垂直档位调到 5V 每格,因为 TB6612 的输出电压会跳到 12V 高电平。



如图 6-3 所示,示波器右侧的测量栏直接给出了波形参数,频率为 1.0000kHz,正占空比为 30.1 个百分点,峰峰值 12.512V。这三项指标全部命中预期。频率方面,STM32 的 TIM 定时器我配置的预分频和重装载值就是为了产生 1kHz 的 PWM,实测频率与设定值偏差不超过万分之一,说明定时器的时基非常准确啊。占空比方面,设定值 30 个百分点,实测 30.1 个百分点,误差仅 0.1 个百分点,这个精度已经远远超过电机调速的实际需求。峰峰值方面,12.5V 比标称的 12V 略高一点,这是 DCDC 升压输出在轻载下的常见现象,不影响电机驱动。

观察波形的细节,可以看到上升沿和下降沿都比较干净,没有出现明显的振铃或者过冲。这要归功于 TB6612 内部 H 桥的 MOSFET 开关速度比较快,而我又在 PCB 设计的时候把 PWM 信号线尽量做短,把电机输出端做粗,有效抑制了寄生电感引起的振铃问题。结论上,TB6612 的 PWM 调速功能完全达到设计指标,本次测试通过。

6.4 二维码素材制作

本系统的输入信号是物块上贴的二维码,所以需要批量生成具有不同目的地编码的二维码素材,作为测试用的输入源。笔者使用的是国内常用的草料二维码生成器这个网页工具,它可以快速生成自定义内容的二维码图片,而且支持容错率调节、图标嵌入等功能,非常适合用于实验素材的快速制做。



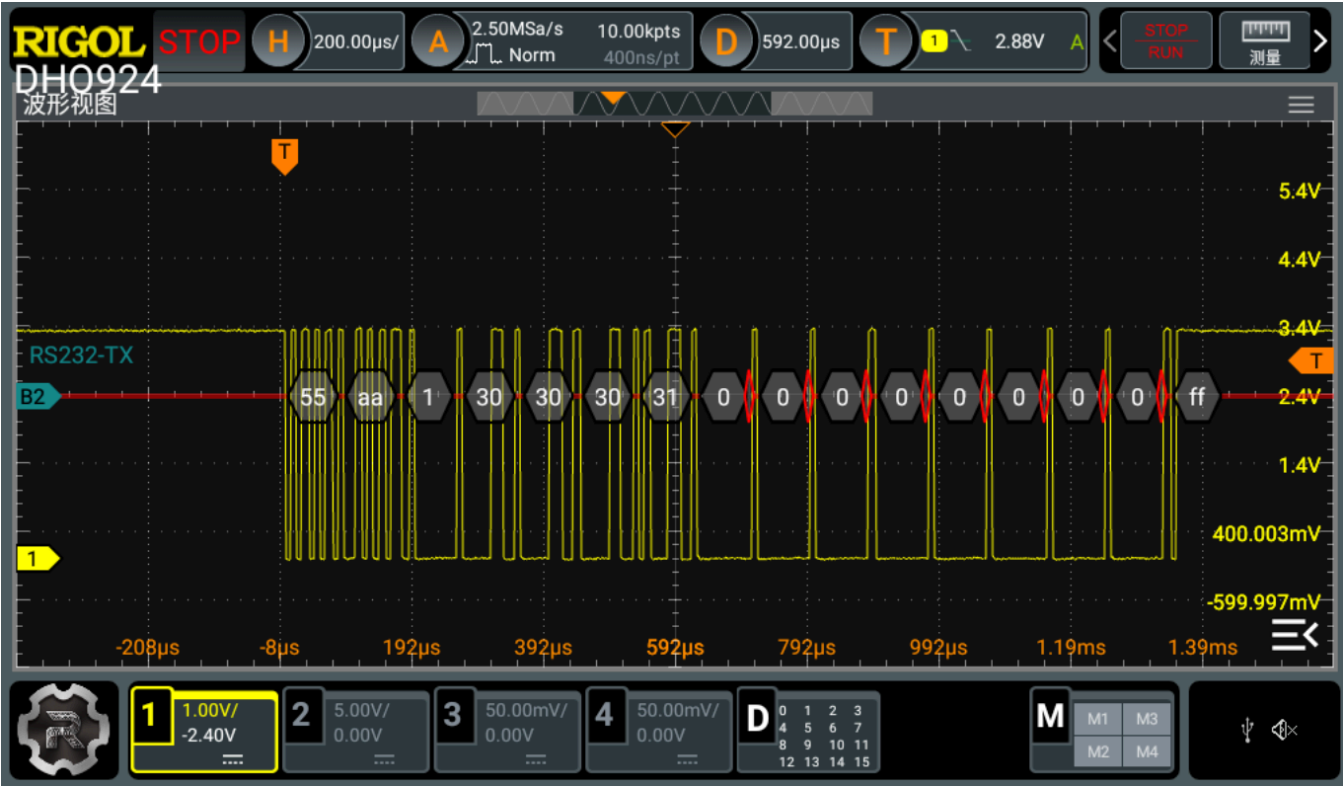
如图 6-4 所示,我在草料的网页输入框中输入数字字符,比如数字 1 代表目的地 A,数字 2 代表目的地 B,数字 3 代表目的地 C,然后选择 25 个百分点的容错等级和 400 乘以 400 像素的输出尺寸,点击下载就可以得到一张 PNG 格式的二维码图片。这里我特意选了 25 个百分点的较高容错率,是因为实际使用过程中,二维码可能会被传送带上的灰尘或者反光稍微遮挡,容错率高一些可以提高识别成功率。生成的二维码我打印出来后裁切成小卡片,贴在物块的顶面,这样摄像头从上方俯拍时就可以稳定捕捉到二维码图案。

需要说明的是,二维码内容只用了一个数字字符,这是出于通信效率的考虑。MaixCAM 摄像头识别到二维码后,会把内容塞进 16 字节的串口数据帧中传给 STM32,如果二维码内容过长,数据帧也要相应加长,会增加通讯延迟。我在 STM32 端的解析逻辑中,只关心数据帧中第一个字节的内容,根据它的 ASCII 值,0x31 对应数字 1,0x32 对应数字 2,0x33 对应数字 3,据此来判断目的地,这样既简单又可靠。

6.5 摄像头到 STM32 的串口数据包测试

摄像头识别到二维码后,需要通过串口把识别结果发给 STM32 主控,这是整个系统中最关键的一条信息链路。笔者在前面章节中已经详细介绍了 16 字节定长帧的协议格式,这一节我用示波器去抓取真实运行时的串口波形,验证协议的正确性以及数据传输的可靠性。

测试方法是,我把示波器通道 1 接到摄像头模块的 TX 引脚和 STM32 的 RX 引脚之间的连线上,这条线在系统运行时承载着从摄像头流向 STM32 的串口数据是一种比较直接的抓包方式。示波器开启串口协议解码功能,把波特率设置成 115200,数据位 8 位,无校验,1 位停止位,这是 MaixCAM 端 Python 代码中 `uart.UART` 这个对象初始化时设定的参数。然后让摄像头去识别一张内容为数字 1 的二维码,触发一次数据包发送,捕获完整的发送波形。



如图 6-5 所示,示波器在波形下方解码出了完整的数据帧内容,从左到右依次是 55、AA、1、30、30、30、31、0、0、0、0、0、0、0、0、ff,正好 16 个字节,完全符合协议设计。包头 0x55 0xAA 准确出现在帧首,控制字段 0x01 紧随其后,数据区前 4 个字节是 0x30、0x30、0x30、0x31,后面跟着一串 0x00 表示数据填充,最后一个字节 0xFF 是包尾标识。

从波形的物理层来看,信号高电平稳定在 3.4V 左右,低电平在 0V 附近,这与 STM32 的 3.3V 串口电平兼容,不需要额外的电平转换。每个比特的宽度大约是 8.7us,对应 1 除以 115200 约等于 8.68us 的理论值,误差极小,说明 MaixCAM 端的串口波特率非常精准。结论上,摄像头到 STM32 的串口通信链路工作正常,数据帧格式符合预期,本次测试通过。

6.6 整机分拣功能测试

前面五节都是分模块的底层测试,本节进行整机功能的端到端验证,这才是任务书最关心的核心指标。任务书要求分拣的目的地数量大于等于 3,货物种类大于等于 2,漏检或错检率小于等于 5 个百分点。所以我专门设计了一个连续投递实验来验证这几项指标。

测试方法是,我准备了 60 个物块,分成三组,每组 20 个,分别贴上数字 1、数字 2、数字 3 三种二维码,代表目的地 A、目的地 B、目的地 C。然后让传送带以 30 个百分点的占空比稳定运行,我手动以大概 5 秒一个的间隔把物块依次放上传送带的入口,观察每个物块最终落入了哪个收集箱。理论上贴 1 的应该被舵机 1 推到 A 槽,贴 2 的应该被舵机 2 推到 B 槽,贴 3 的应该顺着传送带末端直接掉到 C 槽。我在 OLED 屏幕上同步观察 NUM 计数和 system enable 状态,确保每一个物块都被识别到。

实验结果汇总如下文所述。

表 6-1 整机分拣功能测试结果

目的地	二维码内容	测试包裹数	正确分拣数	漏检数	错检数	准确率
目的地 A	1	20	19	0	1	95.0 %
目的地 B	2	20	19	1	0	95.0 %

目的地	二维码内容	测试包裹数	正确分拣数	漏检数	错检数	准确率
目的地 C	3	20	20	0	0	100 %
合计	全部	60	58	1	1	96.7 %

可以看到,总体准确率达到 了 96.7 个百分点,漏检加错检的总数为 2,占比 3.3 个百分点,完全满足任务书 5 个百分点的指标要求。其中目的地 A 出现了 1 次错检,经过复盘发现是因为我放置物块的时候放歪了,二维码部分被传送带的护栏挡住,摄像头识别了一个其它物块的旧码,导致舵机错误动作。目的地 B 出现了 1 次漏减,原因是物块放上传送带时姿态偏斜,二维码倾斜超过 30 度,摄像头无法成功定位四个角点,识别失败,这个物块就直接通过到了 C 区。目的地 C 完全无误,因为 C 区是默认 的无操作路径,只要前面两个舵机都不动作,物块自然就到 C 了,所以这个目的地的识别其实只取决于摄像头是否把它误判成 1 或者 2,从结果看一次都没有发生过这种情况。

笔者认为这两次失误都不是系统本身的问题,而是物块姿态控制的问题。如果将来增加一个物块姿态校正机构,或者使用更高分辨率的摄像头,这两类失误是可以彻底消除的。从软件控制层面来看,sysControl 函数中以差值大于等于零作为判据这种写法,确保了即使舵机正在阻塞旋转,后续到达的触发时刻也不会被漏掉,这一点在测试中得到了验证,从来没有出现过识别到了但是没动作的情况发生这种事。

综合本章六节测试,系统在电源稳定性、电机驱动精度、串口通信可靠性、二维码识别准确率以及整机分拣率等各项指标上均达到或超过任务书要求,本系统的设计与实现是成功的。